BỘ QUẢN LÝ TRÌNH BÀY

Có năm bộ quản lý trình bày:

- Flow Layout
- Border Layout
- Grid Layout
- Gridbag Layout
- Null Layout

GV: Võ Tấn Dũng

FLOW LAYOUT

Đối với một container trình bày theo kiểu FlowLayout thì:

- Mặc định của Jpanel
- Các component gắn vào được sắp xếp theo thứ tự từ trái sang phải và từ trên xuống dưới.
- Các component có kích thước như mong muốn.
- Nếu chiều rộng của Container không đủ chỗ cho các component thì chúng tự động tạo ra một dòng mới.
- FlowLayout thường được dùng để để sắp xếp các button trong 1 panel.
- Chúng ta có thể điều chỉnh khoảng cách giữa các component.

GV: Võ Tấn Dũng

FLOW LAYOUT

Đặc điểm hoạt động

- **Xếp nối tiếp:** Các component được đặt cạnh nhau trên cùng một hàng. Khi hết chiều rộng cửa sổ, các component tiếp theo sẽ tự động xuống hàng mới.
- **Tự điều chỉnh theo kích thước chuẩn:** FlowLayout tôn trọng kích thước ưu tiên (preferredSize) của từng component thay vì kéo giãn chúng.
- **Căn chỉnh dòng:** Mặc định các thành phần trên mỗi dòng sẽ được căn giữa (FlowLayout.CENTER).

GV: Võ Tấn Dũng

FLOW LAYOUT

Giới thiệu

Hằng số Alignment	Ý nghĩa
`FlowLayout.LEFT`	Căn lề trái các thành phần trên dòng
`FlowLayout.CENTER`	Căn giữa (*mặc định*)
`FlowLayout.RIGHT`	Căn lề phải
`FlowLayout.LEADING`	Căn theo đầu dòng (phụ thuộc hướng ngôn ngữ, thường là trái)
`FlowLayout.TRAILING`	Căn theo cuối dòng

FLOW LAYOUT

Ví dụ:

```
import java.awt.*;
import java.lang.Integer;
class FlowLayoutDemo
{
        public static void main(String args[])
        {
                Frame fr = new Frame("FlowLayout Demo");
                fr.setLayout(new FlowLayout());
                fr.add(new Button("Red"));
                fr.add(new Button("Green"));
                fr.add(new Button("Blue"));
                List li = new List();
                for (int i=0; i<5; i++)
                {
                        li.add(Integer.toString(i));
                 }
```

//xem tiếp ở slide tiếp theo

GV: Võ Tấn Dũng

FLOW LAYOUT

```
                fr.add(li);
                fr.add(new Checkbox("Pick me", true));
                fr.add(new Label("Enter your name:"));
                fr.add(new TextField(20));
                // phương thức pack() được gọi sẽ làm cho cửa sổ
                // hiện hành sẽ có kích thước vừa với kích thước
                // trình bày bố trí những thành phần con của nó.
                fr.pack();
                fr.setVisible(true);
        }
}
```



GV: Võ Tấn Dũng

BORDER LAYOUT

Đối với một container trình bày theo kiểu BorderLayout thì:

- Bộ trình bày khung chứa được chia làm 4 + 1 vùng: NORTH, SOUTH, WEST, EAST và CENTER. (Đông,Tây, Nam, Bắc và trung tâm).
- Bộ trình bày loại này cho phép sắp xếp và thay đổi kích thước của những components chứa trong nó sao cho vừa với 5 vùng ĐÔNG, TÂY, NAM, BẮC, TRUNG TÂM.
- Không cần phải gắn component vào cho tất cả các vùng.
- Các component ở vùng NORTH và SOUTH có chiều cao tùy ý nhưng có chiều rộng đúng bằng chiều rộng vùng chứa.
- Các component ở vùng EAST và WEST có chiều rộng tùy ý nhưng có chiều cao đúng bằng chiều cao vùng chứa.
- Các component ở vùng CENTER có chiều cao và chiều rộng phụ thuộc vào các vùng xung quanh.

GV: Võ Tấn Dũng

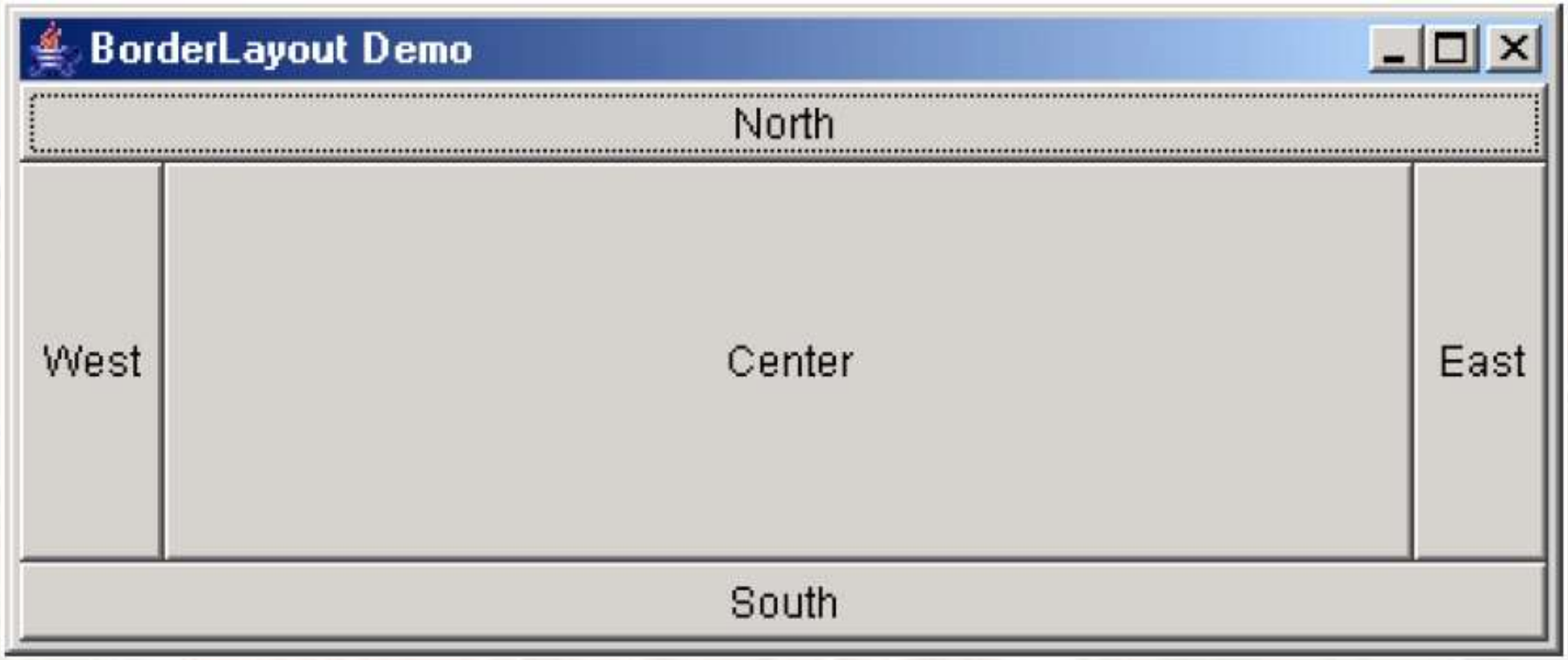
BORDER LAYOUT

Ví dụ:

```
import java.awt.*;
class BorderLayoutDemo extends Frame
{       private Button north, south, east, west, center;
        public BorderLayoutDemo(String sTitle)
        {       super(sTitle);
                north = new Button("North");
                south = new Button("South");
                east = new Button("East");
                west = new Button("West");
                center = new Button("Center");
                this.add(north, BorderLayout.NORTH);
                this.add(south, BorderLayout.SOUTH);
                this.add(east, BorderLayout.EAST);
                this.add(west, BorderLayout.WEST);
                this.add(center, BorderLayout.CENTER);
        }
        public static void main(String args[])
        {       Frame fr = new BorderLayoutDemo ("Border Layout Demo");
                fr.pack();
                fr.setVisible(true);
        }
}
```


GV: Võ Tấn Dũng

BORDER LAYOUT

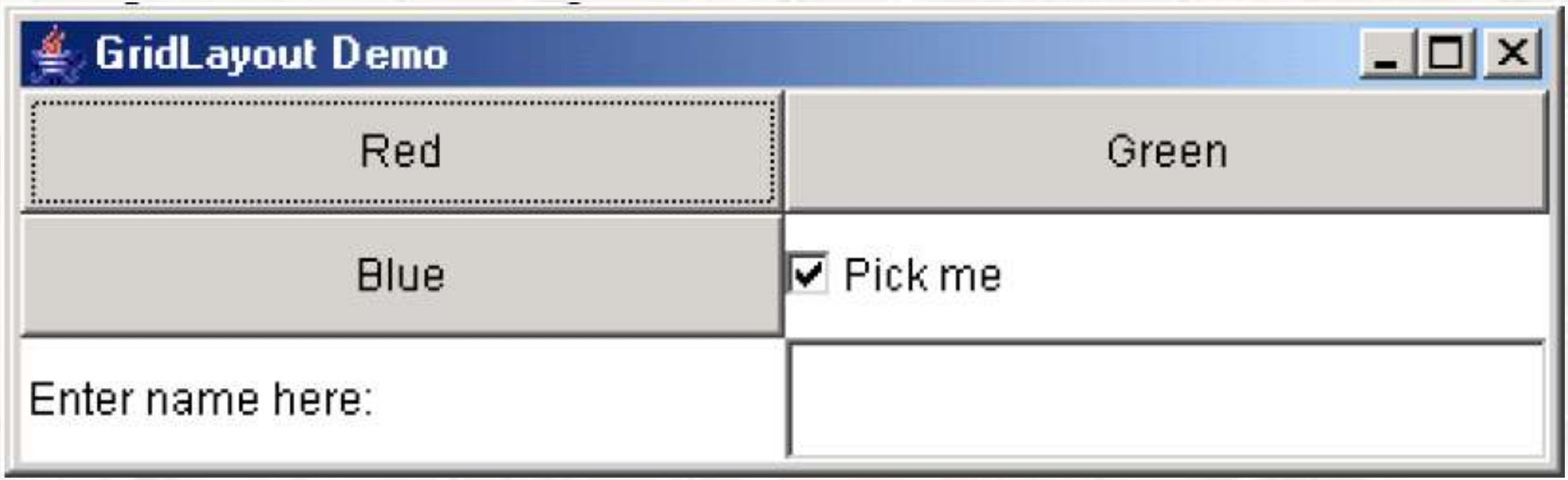


GV: Võ Tấn Dũng

GRID LAYOUT

Đối với một container trình bày theo kiểu GridLayout thì:

- Bộ trình bày tạo một khung lưới vô hình với các ô bằng nhau.
- Các đối tượng sẽ đặt vừa kích thước với từng ô đó. Thứ tự sắp xếp từ trái qua phải và từ trên xuống dưới.



GV: Võ Tấn Dũng

GRID LAYOUT

Ví dụ:

```
import java.awt.*;
public class GridLayoutDemo{
        public static void main(String arg[])        {
                Frame f = new Frame("GridLayout Demo");
                f.setLayout(new GridLayout(3,2));
                f.add(new Button("Red"));
                f.add(new Button("Green"));
                f.add(new Button("Blue"));
                f.add(new Checkbox("Pick ", true));
                f.add(new Label("Enter nam ere:"));
                f.add(new TextField());
                f.pack();
                f.setVisible(true);
        }
}
```

3 phương thức nạp chồng

GRID LAYOUT

Các phương thức khởi tạo của GridLayout

- GridLayout(): Tạo một lưới mặc định với 1 hàng và 1 cột cho mỗi thành phần.
- GridLayout(int rows, int columns): Tạo một lưới với số hàng (rows) và số cột (columns) được chỉ định, không có khoảng cách giữa các ô.
- GridLayout(int rows, int columns, int hgap, int vgap): Tạo một lưới với số hàng, số cột, kèm theo khoảng cách theo chiều ngang (hgap) và chiều dọc (vgap) giữa các thành phần.

3 phương thức nạp chồng

GV: Võ Tấn Dũng

GRIDBAG LAYOUT

Đối với một container trình bày theo kiểu GridBagLayout thì:

- Các componets khi được đưa vào khung chứa sẽ được trình bày trên 1 khung lưới vô hình tương tự như GridLayout.
- Tuy nhiên khác với GridLayout kích thước các đối tượng không nhất thiết phải vừa với 1 ô trên khung lưới mà có thể là 2, 3 ô hay nhiều hơn tùy theo các ràng buộc mà ta chỉ định thông qua đối tượng GridBagConstraints.

GV: Võ Tấn Dũng

GRIDBAG LAYOUT

Lớp **GridBagConstraints** dẫn xuất từ lớp Object.

Lớp GridBagConstraints dùng để chỉ định ràng buộc cho những components trình bày trong khung chứa container theo kiểu GridBagLayout.

- **gridx, gridy**: vị trí ô của khung lưới vô hình mà ta sẽ đưa đối tượng con vào ô **gridwidth, gridheight**: kích thước hay vùng trình bày cho đối tượng con.
- **Insets**: là một biến đối tượng thuộc lớp Inset dùng để qui định khoảng cách biên phân cách theo 4 chiều (trên, dưới, trái, phải).
- **weightx, weighty**: chỉ định khoảng cách lớn ra tương đối của các đối tượng con với nhau

GV: Võ Tấn Dũng

GRIDBAG LAYOUT

Ví dụ:

```
import java.awt.*;
public class GridBagLayoutDemo
{
        public static void main(String arg[])
        {
                Frame f = new Frame("GridBagLayout Demo");
                // Thiet lap layout manager
                // Tao doi tuong rang buoc cho cach trinh bay
                // GridBagLayout.
                GridBagLayout layout = new GridBagLayout();
                GridBagConstraints constraints = new
                GridBagConstraints();
                f.setLayout(layout);
                // Tao ra 9 nut nhan
                String[] buttName = {"Mot", "Hai", "Ba", "Bon", "Nam", "Sau", "Bay",
                                                                "Tam", "Chin"};
                Button[] buttons = new Button[9];
                for(int i=0;i<9;i++)
                {
                        buttons[i] = new Button (buttName[i]);
                }
```

//xem tiếp ở slide tiếp theo

GV: Võ Tấn Dũng

GRIDBAG LAYOUT

```
// Rang buoc cac nut nhan cach nhau 2 pixel
constraints.insets = new Insets(2,2,2,2);
// Qui dinh cac nut nhan se thay doi kich thuoc
// theo ca 2 chieu
constraints.fill = GridBagConstraints.BOTH;
// Rang buoc cho nut nhan thu 1
constraints.gridx = 1;          constraints.gridy = 1;
constraints.gridheight = 2;     constraints.gridwidth = 1;
layout.setConstraints(buttons[0], constraints);
// Rang buoc cho nut nhan thu 2
constraints.gridx = 2;          constraints.gridy = 1;
constraints.gridheight = 1;     constraints.gridwidth = 2;
layout.setConstraints(buttons[1], constraints);
// Rang buoc cho nut nhan thu 3
constraints.gridx = 2;          constraints.gridy = 2;
constraints.gridheight = 1;     constraints.gridwidth = 1;
layout.setConstraints(buttons[2], constraints);
// Rang buoc cho nut nhan thu 4
constraints.gridx = 1;          constraints.gridy = 3;
constraints.gridheight = 1;     constraints.gridwidth = 2;
layout.setConstraints(buttons[3], constraints);
```

//xem tiếp ở slide tiếp theo

GRIDBAG LAYOUT

- **Khoảng cách và Lề (Padding & Insets)**
- insets: Khoảng cách lề (cách lề ngoài) của thành phần so với các cạnh của ô. Sử dụng đối tượng Insets(top, left, bottom, right).
- ipadx: Khoảng cách đệm bên trong (internal padding) theo **chiều ngang** (làm thành phần rộng ra).
- ipady: Khoảng cách đệm bên trong (internal padding) theo **chiều dọc** (làm thành phần cao lên).

GV: Võ Tấn Dũng

GRIDBAG LAYOUT

- fill: Nhận các giá trị:
  - GridBagConstraints.NONE: Giữ nguyên kích thước mặc định (mặc định).
  - GridBagConstraints.HORIZONTAL: Giãn hết chiều ngang của ô.
  - GridBagConstraints.VERTICAL: Giãn hết chiều dọc của ô.
  - GridBagConstraints.BOTH: Giãn hết cả chiều ngang và chiều dọc.

GV: Võ Tấn Dũng

GRIDBAG LAYOUT

Vị trí trên lưới (Grid Position)

- gridx: Vị trí cột của thành phần (bắt đầu từ 0).
  - Giá trị GridBagConstraints.RELATIVE nghĩa là đặt ngay sau thành phần trước đó.
- gridy: Vị trí hàng của thành phần (bắt đầu từ 0).
  - Giá trị GridBagConstraints.RELATIVE nghĩa là đặt ngay dưới thành phần trước đó.

GV: Võ Tấn Dũng

GRIDBAG LAYOUT

Kích thước ô chiếm dụng (Grid Size)

•gridwidth: Số lượng **cột** mà thành phần đó sẽ chiếm (mặc định là 1). Dùng GridBagConstraints.REMAINDER nếu muốn thành phần đó chiếm hết phần còn lại của hàng.

•gridheight: Số lượng **hàng** mà thành phần đó sẽ chiếm (mặc định là 1). Dùng GridBagConstraints.REMAINDER nếu muốn thành phần đó chiếm hết phần còn lại của cột.

GV: Võ Tấn Dũng

GRIDBAG LAYOUT

- anchor: Nhận các giá trị theo hướng la bàn như:
  - GridBagConstraints.CENTER (ở giữa - mặc định)
  - GridBagConstraints.NORTH (trên cùng giữa), SOUTH (dưới cùng giữa), EAST (phải giữa), WEST (trái giữa).
  - GridBagConstraints.NORTHWEST, NORTHEAST, SOUTHWEST, SOUTHEAST (các góc).

GV: Võ Tấn Dũng

GRIDBAG LAYOUT

```
        // Rang buoc cho nut nhan thu 5
        constraints.gridx = 3;              constraints.gridy = 2;
        constraints.gridheight = 2;         constraints.gridwidth = 1;
        layout.setConstraints(buttons[4], constraints);
        // Rang buoc cho nut nhan thu 6
        constraints.gridx = 4;              constraints.gridy = 1;
        constraints.gridheight = 3;         constraints.gridwidth = 1;
        layout.setConstraints(buttons[5], constraints);
        // Tu nut thu 7 tro di khong can rang buoc
        // thay vi doi kich thuoc
        constraints.fill = GridBagConstraints.NONE;
        // Rang buoc cho nut nhan thu 7
        constraints.gridx = 1;              constraints.gridy = 4;
        constraints.gridheight = 1;         constraints.gridwidth = 1;
        constraints.weightx = 1.0;
        layout.setConstraints(buttons[6], constraints);
        // Rang buoc cho nut nhan thu 8
        constraints.gridx = 2;              constraints.gridy = 5;
        constraints.gridheight = 1;         constraints.gridwidth = 1;
        constraints.weightx = 2.0;
        layout.setConstraints(buttons[7], constraints);
//xem tiếp ở slide tiếp theo
```

GRIDBAG LAYOUT

```
            // Rang buoc cho nut nhan thu 9
            constraints.gridx = 3;
            constraints.gridy = 6;
            constraints.gridheight = 1;
            constraints.gridwidth = 1;
            constraints.weightx = 3.0;
            layout.setConstraints(buttons[8], constraints);
            // Dua cac nut nhan khung chua chuong trinh
            for (int i=0;i<9;i++)
                        f.add(buttons[i]);
            f.pack();
            f.setVisible(true);
      }
}
```



GV: Võ Tấn Dũng

NULL LAYOUT

- Một khung chứa được trình bày theo kiểu Null Layout có nghĩa là người lập trình phải tự làm tất cả từ việc qui định kích thước của khung chứa, cũng như kích thước và vị trí của từng đối tượng component trong khung chứa.
- Để thiết lập cách trình bày là Null Layout cho một container ta chỉ việc gọi phương thức setLayout(**null**) với tham số là **null.**

GV: Võ Tấn Dũng

NULL LAYOUT

Một số phương thức của lớp trừu tượng Component dùng để định vị và qui định kích thước của component khi đưa chúng vào khung chứa trình bày theo kiểu kiểu tự do:

- public void setLocation(Point p)
- public void setSize(Dimension p)
- public void setBounds(Rectangle r)

Ví dụ:

- MyButton.setSize(new Dimension(20, 10));
- MyButton.setLocation(new Point(10, 10));
- MyButton.setBounds(10, 10, 20, 10);

GV: Võ Tấn Dũng

NULL LAYOUT

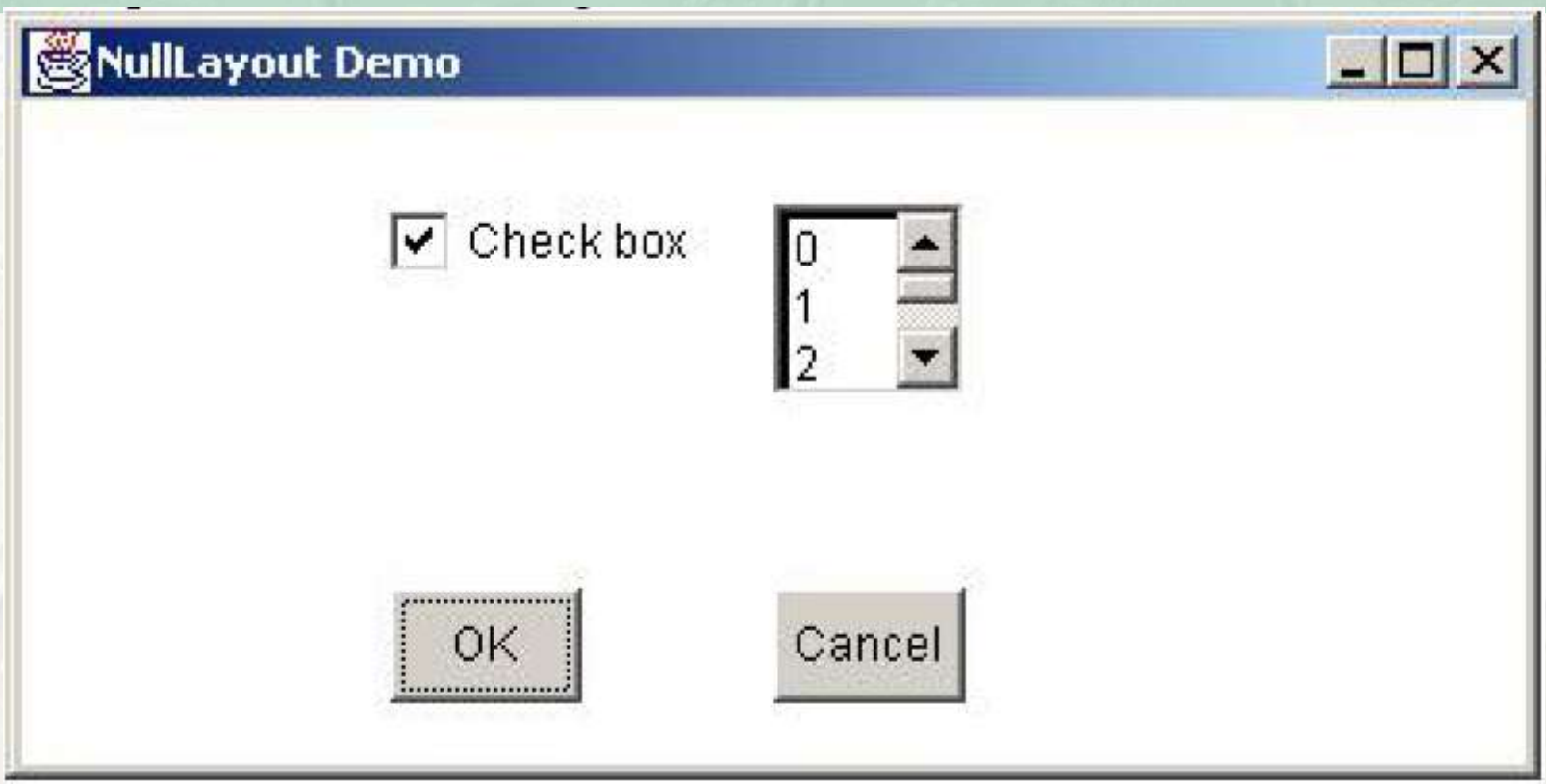
```
import java.awt.*;
class NullLayoutDemo{
        public static void main(String args[])       {
                Frame fr = new Frame("NullLayout Demo");
                fr.setLayout(null);
                Button buttOk = new Button("OK");
                buttOk.setBounds(100, 150, 50, 30);
                Button buttCancel = new Button("Cancel");
                buttCancel.setBounds(200, 150, 50, 30);
                Checkbox checkBut = new Checkbox("Check box", true);
                checkBut.setBounds(100, 50, 100, 20);
                List li = new List();
                for (int i=0; i<5; i++)                  {
                        li.add(Integer.toString(i));
                }
                li.setBounds(200, 50, 50, 50);
                fr.add(buttOk);
                fr.add(buttCancel);
```


//xem tiếp ở slide tiếp theo

GV: Võ Tấn Dũng

NULL LAYOUT

```
                fr.add(checkBut);
                fr.add(li);
                fr.setBounds(10, 10, 400, 200);
                fr.setVisible(true);
        }
}
```



GV: Võ Tấn Dũng

HẾT
CHƯƠNG 4